

Command Flow: App → Cloud → Drone

How a natural-language command like "fly to the chair and back" gets from the iOS app to a MAVLink message on the flight controller.

The NLP boundary lives in **AWS Lambda**, not on the drone. The app ships raw text; Claude (via Bedrock) turns it into a phased mission; the drone executes each phase with on-device VLM + Nav2 + MAVLink.

1. iOS — user types text

`DroneChatView.swift:75` takes the input. `sendMessage()` calls `APIClient.swift:165`:

```
POST /drones/{droneId}/conversations/{conversationId}/messages
Authorization: Bearer <Cognito idToken>
Content-Type: application/json

{"message": "fly to the chair and back"}
```

The app does not publish to MQTT directly — it is pure REST through API Gateway.

2. AWS — Lambda interprets via Claude

API Gateway routes to `message_handler` in `aws/src/conversations.py:938`. It:

1. Auths the user against the drone (`drone_id` ownership check).
2. Appends the user message to the DynamoDB conversation log.
3. Retrieves conversation history.
4. Branches on drone capability (`has_vlm` , `variant`):
5. **VLM-capable (AGX):** `call_mission_agent()` — Claude returns a multi-phase mission JSON.
6. **Legacy (Nano/NX):** `call_agent()` — simpler goal response.

Claude 4.5 Sonnet (Bedrock) returns:

```
{
  "action": "mission",
  "mission": {
    "phases": [
      {"objective": "Locate the chair in current environment", "success": "Chair identified in view"},
      {"objective": "Navigate to the chair", "success": "Drone positioned at chair location"},
      {"objective": "Return to start position", "success": "Back at launch point"}
    ]
  },
  "message": "I'll fly to the chair and come right back."
}
```

3. AWS → drone — MQTT publish

`conversations.py:1047` calls `publish_to_drone()` against AWS IoT Core.

- **Topic:** `drone/{droneId}/chat/{conversationId}/command`

- **QoS:** 1 (at-least-once)
- **Payload:**

```
{
  "action": "mission",
  "mission_id": "...",
  "phases": [ ... ],
  "conversation_id": "...",
  "original_message": "fly to the chair and back"
}
```

4. Drone — daemon receives and dispatches

`daemon.py:1559` subscribes to `drone/{DRONE_ID}/chat/+/command` over MQTT (mutual TLS, certs in `certs/`).

`on_chat_command()` at `daemon.py:956` sees `action == 'mission'`, constructs a `Mission` and spawns a `MissionLoop` on its own thread (~line 1001).

5. Drone — VLM perception loop per phase

`reasoning_loop.py:177` iterates phases until `mission.is_complete()`:

1. **Perceive:** grab the current camera frame.
2. **Decide:** Qwen3-VL on the AGX produces an `ActionType` — e.g. `NAVIGATE_TO_POINT(px, py)`, `LOOK_AROUND`, `PHASE_COMPLETE`.
3. **Act:** dispatch to the navigation stack or advance the phase.

The drone never sees the string "chair" as text — it sees a phase **objective**, and the on-device VLM grounds "chair" against the live camera frame.

6. Nav2 — pixel → 3D waypoint

When the VLM emits `NAVIGATE_TO_POINT`, `nav2_bridge.py:106`:

1. Deprojects the pixel using the depth map.
2. Transforms camera frame → body frame → map frame.
3. Sends a `NavigateToPose` action goal to ROS2 Nav2 for path planning + obstacle avoidance.

7. MAVLink — final wire commands to the flight controller

`drone_sdk.py:418` handles arm/takeoff/land:

```
# Set GUIDED mode, then:
m.mav.command_long_send(target_system, target_component,
    mavutil.mavlink.MAV_CMD_COMPONENT_ARM_DISARM, # 400
    0, 1, 0, 0, 0, 0, 0, 0)
m.mav.command_long_send(target_system, target_component,
    mavutil.mavlink.MAV_CMD_NAV_TAKEOFF, # 22
    0, 0, 0, 0, 0, 0, 0, altitude_m)
```

Per waypoint (`drone_sdk.py:530`):

```
m.mav.set_position_target_global_int_send(
    0, 1, 1,
    mavutil.mavlink.MAV_FRAME_GLOBAL_RELATIVE_ALT_INT,
    0b0000111111111000, # position-only mask
    int(lat * 1e7), int(lon * 1e7), alt,
    0, 0, 0, 0, 0, 0, 0, 0)
```

Return-to-launch is the same `SET_POSITION_TARGET_GLOBAL_INT` pointed back at takeoff coords, followed by `MAV_CMD_NAV_LAND` (21).

Summary

Hop	Where	Transport	Payload
App → API	APIClient.swift:165	HTTPS	{"message": "..."}
Claude planning	conversations.py:975	Bedrock	text → phase JSON
API → drone	conversations.py:1047	MQTT drone/{id}/chat/{conv}/command	mission JSON
Phase execution	reasoning_loop.py:177	on-device	VLM action stream
Waypoint planning	nav2_bridge.py:106	ROS2	NavigateToPose
To autopilot	drone_sdk.py:530	MAVLink (serial)	SET_POSITION_TARGET_GLOBAL_INT